

星云作业 - 银河平台部署问题定位工具(学生)

一、课题背景

银河平台是金山云的私有云平台，涵盖计算、网络、存储、智算等全量私有化云能力。在客户环境进行部署时，虽然有测试环境完善的测试、以及自动化的部署验证工具，但是面临复杂的客户环境（不同厂商的硬件、利旧、第三方存储设备等），经常会出现部署后各类疑难问题。此类问题往往需要运维 + 研发同学人工排查。此外，部分客户环境出于安全考虑，没有直接的互联网网络连接，无法且不接受公共 AI 工具进行排查分析。

客户环境痛点场景示例：

- 客户 A 利旧服务器混搭不同厂商硬件，部署后存储挂载反复偶发不稳，排查耗时 2 天，最终发现是某厂商 RAID 卡固件版本与系统内核存在兼容性问题
- 客户 B 为敏感环境，完全断网（公网），运维人员只能靠现场电话 / 语音沟通 + 内网登录手动翻日志定位问题，平均排查周期是联网环境的 3-5 倍
- 客户 C 使用第三方 NAS 存储，容器环境下网络配置与平台默认参数冲突，人工排查遗漏关键配置项导致出现隐性故障

你需要构建一个**面向银河平台部署场景、可在断网客户环境运行的部署问题定位工具**，把原来"运维手工翻日志 + 靠经验猜原因"的过程，变成"自动采集 + 结构化推理 + 可控修复建议"的流程

二、目标

工具的典型使用者是**专业的运维人员**，他们通过 SSH 登录到客户服务器（通常无图形界面）操作，需要在断网环境下独立完成"环境识别 → 信息采集 → 根因分析 → 修复建议 → 人工确认 → 执行 → 验证"的完整闭环。

目标：实现一个跑通上述闭环、能在断网环境离线运行、且对生产环境写操作有严格人工审核防线的命令行工具。

三、硬性红线约束（必须满足，任一不满足则视为课题未完成）

红线 1：运行阶段零公网依赖（离线可用）

- "离线"指运行环境无公网出站连接；内网 DNS、内网镜像仓库、内网文件服务器可用。

- 部署阶段允许**一次性**离线导入（U 盘 / 移动硬盘 / 内网传输 / 内网 Docker Registry 推送），但**运行阶段不得有任何公网依赖**。
- 核心功能（模型推理、环境识别、诊断分析、修复建议、安全审核）必须能在断网后正常执行。
- 代码与配置中**不得硬编码任何外网地址**（如 `api.openai.com`、`huggingface.co`）。
- 验证方式：断开被测机器公网出站，重启系统，执行一次完整诊断 - 修复流程，检查日志 / 代码中是否存在对外网域名的请求。

红线 2：生产环境写操作必须人工显式确认

- 任何修改生产环境的操作（执行命令、修改配置、重启服务）执行前**必须有人类显式确认**，系统不得自动执行任何写操作。
- 确认必须通过**专用交互流程**完成，**不得由 LLM 解析用户自然语言来判定"确认"**（否则 Prompt 注入可绕过审核）。
- 用户拒绝时不执行且不反复要求确认。
- 只读 / 纯诊断操作无需确认。

红线 3：必须可端到端演示

- 必须能用真实或模拟数据走通"问题报告 → 修复完成"的完整流程，不接受纯 PPT 或截屏方案。
- "能运行"不等于"能工作"：系统可启动但核心功能全部报错，同样不满足。
- 受硬件限制无法现场演示的环节，可用录屏 + 代码走查补充，但不能完全替代现场实操。

四、必做功能需求

4.1 模型离线部署与运行（REQ-A-01）

本需求已整合原 REQ-A-02（模型离线分发）与 REQ-A-03（运行资源声明）的内容。

- **前置依赖**：无
- **描述**：客户环境无公网，传统模型下载方式不可用。需提供一种方案，使小参量模型可在完全离线环境下完成部署并启动推理服务；部署过程应最大限度减少手动配置步骤，使非 AI 专业运维人员也能独立完成；同时显式声明运行所需最低硬件，并在启动前自动预检。
- **"小参量模型"参考范围**：约 7~12B 参数的小规格开源模型（如 Qwen3.5、Gemma4 系列模型）。具备 GPU 时可选 9B 以上级，纯 CPU 环境建议 3~9B 级。具体选型由你决定，关键是能在目标硬件上离线流畅推理
- **验收标准**：
 - 无公网出站连接的环境中，系统能完成部署并启动推理服务。

- 部署完成后执行一次健康检查，能返回推理服务就绪状态。
- 部署过程不需要编辑配置文件或手动编译依赖；首次部署后重启无需重新导入模型文件。
- 文档中明确描述模型文件**离线导入的完整流程**（含具体操作步骤与所需介质），并提供模型版本**离线更新替换流程**。
- 文档中明确列出最低硬件配置：CPU 核数、GPU 型号与显存（如适用）、内存容量、磁盘空间。
- 部署工具在启动前检测当前硬件是否满足最低要求，不满足时给出明确提示并**拒绝启动**。
- **实现指引：**可选预打包 Docker 镜像（含模型 + 运行时）、离线安装脚本 + 模型文件压缩包、或 Ollama 离线导入（`ollama create` 从本地 GGUF 文件）。模型文件导入介质不限（U 盘 / 内网文件服务器 / 内网 Registry），关键是流程可操作且文档化。资源预检可在部署脚本中加入：检测 CPU 核数、GPU 显存、可用内存、磁盘空间，不满足则退出并提示具体差距。

4.2 环境感知

REQ-B-01 运行环境类型自动识别

- **前置依赖：**无
- **描述：**银河平台可能部署在裸金属、虚拟机（VM）或容器中，不同环境下诊断策略差异很大（如容器内无法直接访问宿主机硬件信息）。系统需自动识别当前运行环境类型，并据此选择对应诊断路径。
- **验收标准：**
 - 能自动识别裸金属、VM、容器三种环境类型。
 - 识别结果对用户可见（输出中声明"当前环境为 XXX"）。
 - 识别逻辑在三种环境中均能正确执行。
- **实现指引：**通过系统特征组合检测，如容器检测（`/ .dockerenv` 文件或 `cgroup` 信息）、VM 检测（`systemd-detect-virt` 或 DMI 信息）。不同环境可组合检测以提高准确度。

REQ-B-02 异构软硬件信息采集

- **前置依赖：**REQ-B-01
- **描述：**客户环境混搭不同厂商硬件、利旧设备、第三方存储。系统需采集这些异构设备的关键信息，作为后续诊断与适配建议的输入。
- **验收标准：**
 - 能采集硬件基本信息：CPU 型号、内存容量、磁盘类型与容量、RAID 卡型号与固件版本、网卡型号。
 - 能采集第三方存储设备信息：存储类型（SAN / NAS / 本地）、挂载路径、文件系统类型。

- 采集结果以结构化格式输出（JSON / YAML），可供后续分析使用。
- 采集过程不影响生产系统运行（只读操作）。
- **实现指引：**封装一组诊断采集工具，根据 B-01 识别的环境类型选择不同采集策略。裸金属采集完整硬件信息，容器环境采集容器可见信息并提示可能需要宿主机信息补充。

4.3 诊断分析

REQ-C-01 问题诊断信息收集

- **前置依赖：**REQ-B-01、REQ-B-02
- **描述：**根因分析的前提是充足的诊断输入。系统需定义清晰的信息收集机制：主动采集（执行诊断脚本收集日志和系统状态）还是被动接收（用户上传日志或描述问题），以及如何将采集的信息组织为可供推理引擎使用的结构化输入。
- **验收标准：**
 - 提供标准化信息收集入口，支持至少 2 种输入方式（如主动采集 + 用户描述 / 日志上传）。
 - 收集的信息包含：银河平台组件部署状态、相关服务日志、系统资源使用情况、网络连通性。
 - 采集的原始信息经预处理后形成结构化诊断上下文，传递给分析引擎。
 - 采集失败时给出明确提示，不静默忽略。
- **实现指引：**设计一组诊断采集工具（Tool），Agent 根据问题描述决定调用哪些采集工具，采集结果结构化后注入 Prompt 上下文。**命题方提供的领域知识**（组件清单、关键日志路径等）预置在采集工具中。

REQ-C-02 多环境根因分析

- **前置依赖：**REQ-C-01、REQ-B-01
- **描述：**基于 C-01 收集的诊断信息和 B-01 识别的环境类型，对部署 / 运行时问题进行根因分析，给出具体原因或排查方向。裸金属、VM、容器三种环境下故障模式和诊断路径差异大，需覆盖。
- **验收标准：**
 - 对已知故障模式，能定位具体原因（非仅给出模糊方向）。
 - 对未知故障模式，能给出可执行的排查步骤和可能的故障范围。
 - 分析结果区分"已确认"和"推测"，不将猜测表述为确定性结论。
 - 在裸金属、VM、容器三种环境中均能执行分析。
- **实现指引：**Agent 基于诊断上下文推理。常见故障模式可通过规则 / 知识库快速匹配（低成本路径），复杂问题由 LLM 推理（高成本路径）。**命题方提供的故障案例库**可作为规则库初始内容。

REQ-C-03 诊断结论与不确定性声明

- **前置依赖：**REQ-C-02
- **描述：**系统不是万能的。当信息不足或超出知识范围时，必须诚实声明不确定性，而非编造似是而非的结论。运维场景中一个错误的"确认"结论比"不确定"更危险。
- **验收标准：**
 - 当分析结果基于不完整信息时，输出中明确标注"信息不足"并列缺失的关键信息。
 - 当无法定位根因时，输出"未能确定根因"并提供后续排查建议，不编造确定性结论。
 - 分析结论中区分"已确认原因"和"推测原因"，两者使用不同的表述方式。
- **实现指引：**在 Prompt 中约束输出格式，要求区分"已确认 / 推测 / 信息不足"三类结论；对 LLM 输出做后处理校验，检查是否存在未标注不确定性的推测性表述。

4.4 修复生成

REQ-D-01 修复命令建议模板（含可编辑与参数化）

- **前置依赖：**REQ-C-02
- **描述：**定位根因后，系统应提供修复操作的命令建议，以模板形式提供，含可调整的参数占位符。建议和生成的脚本不应是只读输出，运维人员需能根据实际环境调整参数、修改步骤顺序、删除不适用步骤。
- **验收标准：**
 - 对已确认的故障，能输出具体的修复命令或操作步骤。
 - 建议中使用参数占位符（如 `<IP>`、`<MOUNT_POINT>`），而非硬编码值。
 - 用户能在 CLI 中直接修改建议的参数值。
 - 用户能跳过或删除建议中的特定步骤。
 - 编辑后的建议在执行前重新经过 REQ-D-03 的错误检测。
 - 编辑操作记录在审计日志中。
 - 每条建议附带安全风险提示（如"此操作将重启 XXX 服务"）。
- **实现指引：**输出带占位符的命令模板，用户在 CLI 中交互式填入参数值或编辑步骤后生成可执行命令。

REQ-D-02 修复脚本 / 代码生成

- **前置依赖：**REQ-D-01
- **描述：**某些修复操作需多步骤编排（如先卸载再修复再重挂载），单条命令无法覆盖。系统应能生成可执行的修复脚本，减少运维人员手动拼接命令出错的风险。
- **验收标准：**
 - 能生成包含多步骤的修复脚本（Bash / Python，至少支持一种）。
 - 生成的脚本包含错误处理逻辑（某步骤失败时不继续执行后续步骤）。
 - 生成的脚本在执行前经过 REQ-D-03 的错误检测。

- 用户能在执行前查看完整脚本内容。
- **实现指引：**LLM 生成脚本骨架，填充已知参数，保留占位符待用户确认。脚本中添加 `set -e` (Bash) 或异常捕获 (Python) 实现基本错误处理。

REQ-D-03 生成代码多维错误检测

- **前置依赖：**REQ-D-02 (同时服务于 D-01 编辑后内容的检测)
- **描述：**LLM 生成的代码 / 脚本可能包含语法错误、安全隐患或不兼容的操作。系统需在用户执行前自动检测这些问题，降低误操作风险。
- **验收标准：**
 - 对生成的脚本 / 代码执行语法检查，语法错误时阻止执行并提示。
 - 检测脚本中的危险操作 (如 `rm -rf /`、`chmod 777`、明文密码)。
 - 检测与环境不兼容的操作 (如在容器环境建议修改 systemd 配置)。
 - 检测结果对用户可见，用户确认是否继续。
- **实现指引：**基于规则的静态分析 (如 ShellCheck 集成、危险命令模式匹配) + Prompt 约束 LLM 不生成已知危险模式。不要求完美检测，但必须有基本防线。

4.5 安全可控

REQ-E-01 人工审核强制拦截

- **前置依赖：**无
- **描述：**运维场景中，任何对生产环境的写操作都必须经过人工显式确认。系统不得自动执行任何修改生产环境的操作 (参见红线 2)。
- **验收标准：**
 - 所有写操作 (执行命令、修改配置、重启服务等) 执行前必须有人类显式确认。
 - 确认操作通过专用交互流程完成，不由 LLM 解析用户自然语言判定"确认"。
 - 用户拒绝确认时，系统不执行且不反复要求确认。
 - 确认与拒绝操作均记录在审计日志中。
- **实现指引：**写操作执行前弹出 CLI 确认提示 (如 `[y/N]`)，确认走专用通道而非 LLM 对话通道；关键操作可要求二次确认或输入特定确认码。

REQ-E-02 危险操作多维防护

- **前置依赖：**REQ-E-01
- **描述：**仅有人工确认还不够。某些操作本身高风险 (删除数据、格式化磁盘、修改关键配置)，即使人工确认也可能因疲劳或信息不对称失误，系统需要多层防线。
- **验收标准：**
 - 维护危险命令 / 操作清单 (如 `rm -rf`、`mkfs`、`dd`、`iptables -F`)，命中清单的操作强制拦截并给出风险说明。

- 对即将执行的脚本进行安全校验（检查是否包含危险模式）。
- 评估操作的影响范围（如"此操作将影响 3 个挂载点、2 个运行中的服务"），将评估结果展示给用户。
- 用户确认危险操作时需额外步骤（如输入 `CONFIRM <操作摘要>`）。
- **实现指引：**危险命令模式库 + 影响范围通过预分析脚本内容得出（如 `grep` 脚本中涉及的服务名和路径）。注意做变量展开检测，避免危险命令被拆成变量绕过。

REQ-E-03 操作快照与一键回滚

- **前置依赖：**REQ-E-01
- **描述：**即使有审核和防护，仍可能有误操作或修复本身引入新问题。系统需在执行修复操作前创建恢复点，使操作可回退。
- **验收标准：**
 - 执行写操作前，系统自动创建恢复快照（至少包含：被修改的配置文件内容、受影响的服务运行状态）。
 - 提供一键回滚命令，可将系统恢复到最近一次快照的状态。
 - 快照元数据可查询（时间、操作内容、影响范围）。
 - 回滚操作本身也需经过 REQ-E-01 的人工审核。
- **实现指引：**修改前备份原始文件到 `.bak` 目录，记录服务状态；回滚命令从备份恢复文件并重启服务；快照元数据存储在本地图文格式文件中。

REQ-E-04 操作留痕与审计日志

- **前置依赖：**REQ-E-01
- **描述：**所有操作决策和执行过程需可追溯。当修复操作引发问题时，需回查"当时谁做了什么决定、基于什么信息"。
- **验收标准：**
 - 每次操作记录：时间戳、操作者、操作内容、操作结果（成功 / 失败 / 回滚）、LLM 分析依据摘要。
 - 审计日志持久化存储（服务重启后不丢失）。
 - 提供日志查询命令。
 - 日志内容不可被 LLM 修改或绕过。
- **实现指引：**操作日志写入本地文件（JSON Lines 格式），Agent 在写日志时使用专用工具而非 LLM 输出，避免日志内容被 Prompt 注入修改。
- **设计警示：**操作日志存在内存中，服务重启后丢失——出事后无法回查历史操作。

4.6 交互与 workflow

REQ-F-01 CLI 交互能力

- **前置依赖：**无
- **描述：**银河平台部署在客户服务器上，运维人员通常通过 SSH 远程操作。系统需提供 CLI 交互方式，使运维人员在无图形界面的服务器环境中也能完整使用系统功能。
- **验收标准：**
 - 提供命令行交互界面，支持诊断、修复建议、审核确认等全部核心操作。
 - 支持交互式参数输入（如编辑修复建议的参数值）。
 - 输出格式对终端友好（使用表格、颜色、缩进等增强可读性）。
 - 提供 `--help` 和命令补全。
- **实现指引：**基于 Python 的 Rich / Textual 库构建 TUI，或使用 Prompt Toolkit 构建交互式 REPL。Web UI 不在本需求范围内。
- **设计警示：**系统只提供 Web 界面，运维人员需额外配置端口转发和浏览器——在断网客户环境中不可行。

REQ-F-02 诊断 - 修复端到端 workflow

- **前置依赖：**REQ-C-01、REQ-C-02、REQ-D-01、REQ-E-01
- **描述：**原始需求列出了多个独立能力，但未定义它们如何串联。系统需定义完整的诊断 - 修复 workflow，使用户从报告问题到问题修复有清晰路径。
- **验收标准：**
 - 系统按核心流程编排：信息收集 → 环境识别 → 根因分析 → 修复建议 → 人工审核 → 执行 → 结果验证。
 - 每个步骤的输入 / 输出定义明确，用户可在任意步骤介入。
 - workflow 状态持久化，用户中断后可从上次位置继续。
 - 简单问题允许跳过部分步骤（如已知故障模式可直接给出修复建议，无需完整推理）。
- **实现指引：**Agent 编排实现 workflow，可以是单 Agent + 状态机，也可以是 Supervisor + 子 Agent 模式。关键是将流程显式化、状态可恢复。
- **设计警示：**各功能独立运行，用户需手动在"诊断模式""修复模式""审核模式"之间切换，且切换时丢失上下文。

REQ-F-03 审核确认交互流程

- **前置依赖：**REQ-E-01、REQ-F-01
- **描述：**REQ-E-01 定义了审核机制的逻辑要求，本需求定义其在 CLI 中的具体交互流程。审核流程是人与系统协作的关键触点，交互设计直接影响安全性和使用效率。
- **验收标准：**
 - 执行写操作前，CLI 展示操作摘要（将执行什么、影响什么、回滚方案），等待用户确认。
 - 用户可执行三种操作：确认执行、拒绝执行、修改参数后重新提交。
 - 确认操作通过键盘输入完成（如输入 `y`），不通过自然语言对话判定。

- 对危险操作，确认流程增加额外步骤（如要求输入 `CONFIRM` 全称）。
- 确认 / 拒绝 / 修改均记录在审计日志中。

五、选做需求

以下为选做，建议在必做需求完成后再考虑，用于提升工具的实用性与可维护性

REQ-X-02 客户知识库集成

- **前置依赖：**REQ-C-01
- **描述：**不同客户环境的历史故障各有特点。系统应支持导入客户特有的故障案例，形成可检索的知识库，使系统能利用客户环境特有经验进行诊断。
- **验收标准：**
 - 支持导入文本格式的故障案例（至少支持 Markdown 或纯文本）。
 - 导入的案例可通过语义检索在诊断过程中被引用。
 - 诊断输出中标注信息来源（来自通用知识还是客户特有案例）。
 - 提供知识库管理命令（导入、列表、删除）。
- **实现指引：**文本分块 + Embedding + 向量存储，检索结果注入诊断 Prompt 上下文。不限定具体向量库方案，需支持离线运行。

REQ-X-04 Agent 推理过程可观测

- **前置依赖：**REQ-F-02
- **描述：**Agent 的推理过程对运维人员应透明——"系统为什么建议这样修复"需要可追溯。这不仅是可观测性，也是审计和信任建设的基础。
- **验收标准：**
 - 每次诊断记录完整的推理链路：调用了哪些工具、得到了什么结果、基于什么逻辑得出结论。
 - 用户可通过命令查看指定诊断任务的推理过程。
 - 推理链路持久化存储，服务重启后可查询。
 - 推理链路内容与诊断结论一致，不存在"结论与依据矛盾"的情况。
- **实现指引：**Agent 每步操作追加到 trace 日志，诊断完成后 trace 可通过命令回放。可集成 LangSmith / Langfuse 或自建简易 trace 模块（需支持离线）。
- **设计警示：**系统输出"建议执行 XXX 命令"，运维人员问"为什么"，系统无法给出推理依据——信任无从建立。

七、典型验证场景

三个场景分别对应裸金属、容器、VM 三种环境，覆盖课题要求的三种运行环境。建议至少准备其中 2 个进行实测验证，含成功与失败案例。

场景 2：容器网络不通（容器环境）

- **场景描述：**银河平台容器组件部署后，容器间网络无法通信，`ping` 不通，服务注册失败。
- **前置条件：**
 - 容器环境（Kubernetes），已部署银河平台容器组件。
 - 部分容器网络插件配置异常。
 - 容器日志中有网络相关错误。
- **操作步骤：**
 - a. 用户报告"容器间网络不通"。
 - b. 系统识别容器环境（REQ-B-01）。
 - c. 系统采集容器网络信息（REQ-B-02）：CNI 配置、iptables 规则、网络插件状态。
 - d. 系统收集诊断信息（REQ-C-01）。
 - e. 系统分析根因（REQ-C-02）。
 - f. 系统生成修复脚本（REQ-D-02）：包含多步骤编排（如重建网络插件 → 重启 CNI → 验证连通性）。
 - g. 系统检测脚本安全性（REQ-D-03）。
 - h. 系统评估影响范围（REQ-E-02）。
 - i. 系统请求确认（REQ-E-01、REQ-F-03），展示影响范围。
 - j. 用户确认执行。
 - k. 系统执行并验证。
- **预期验证点：**
 - 正确识别为容器环境，不执行裸金属专有的硬件检测命令。
 - 采集容器可见信息（CNI 配置、Pod 网络状态），非宿主机硬件信息。
 - 排查方向覆盖 CNI 配置、网络策略、iptables 等容器常见故障点。
 - 生成的脚本包含错误处理（某步骤失败不继续）。
 - 检测脚本中无 `iptables -F` 等危险命令；评估影响范围并展示。
 - 执行前需确认。

场景 3：VM 启动后数据磁盘未识别（VM 环境）

- **场景描述：**银河平台 VM 部署后，数据磁盘不可见，`lsblk` 只显示系统盘，数据盘丢失。
- **前置条件：**

- VM 环境（QEMU-KVM / VMware），已部署银河平台纳管。
 - VM 使用利旧硬件，磁盘控制器类型与默认驱动不匹配。
 - VM 操作系统日志中有磁盘识别相关警告。
 - **操作步骤：**
 - a. 用户报告"数据磁盘未识别"。
 - b. 系统识别 VM 环境（REQ-B-01）。
 - c. 系统采集 VM 磁盘和控制器信息（REQ-B-02）：磁盘控制器类型、驱动模块加载状态。
 - d. 系统分析根因（REQ-C-02）：定位到控制器驱动未加载。
 - e. 系统生成修复脚本（REQ-D-02）：加载驱动模块 → 扫描 SCSI 总线 → 验证磁盘可见。
 - f. 系统检测脚本（REQ-D-03）。
 - g. 系统创建快照（REQ-E-03）。
 - h. 系统请求确认（REQ-E-01）。
 - i. 用户确认执行。
 - j. 系统执行并验证。
 - **预期验证点：**
 - 识别出非标准控制器类型，给出加载对应驱动的建议。
 - 生成的脚本直接可运行，包含 `modprobe` + `rescan-scsi-bus` + 验证步骤。
 - 执行前自动创建快照，快照包含当前系统可恢复状态。
 - 如修复失败，可通过一键命令回滚到快照状态。
-

八、技术约束

8.1 模型协议约束（强制）

- 系统必须通过兼容 **OpenAI Chat Completions API** 的接口调用模型推理。
- `base_url` 必须可配置（支持替换为本地模型服务、vLLM、Ollama、llama.cpp 等）。
- `model` 字段必须可配置。
- 不得硬编码任何外网 API 地址。

8.2 交互约束（强制）

- CLI 为首选交互方式，全部必做功能必须可通过 CLI 完成。
- Web UI 不在必做范围内。

8.3 运行环境约束（强制）

- 目标运行环境为 **Linux (x86_64)**。

- 必须支持无公网出站连接的环境。
- 部署阶段允许一次性离线导入，运行阶段零公网依赖。

8.4 开放选型（不限定）

- 框架选型：LangChain、LlamaIndex、自研 Agent 框架等均可。
- 模型选型：任何可通过 OpenAI 兼容 API 调用的本地小参量模型均可（如 Qwen3.5-7B、DeepSeek-7B 等）。
- 向量存储：任何支持离线运行的方案均可。
- 编程语言：不限，Python 为推荐但非强制。

九、提交物清单

序号	提交物	说明
1	源代码仓库	包含完整可运行的源码 README
2	部署文档	离线部署步骤（含模型 程）、最低硬件要求
3	使用文档	CLI 命令参考、典型使用
4	API 文档	系统对外接口说明（如
5	测试报告	至少 2 个场景的实测结 成功和失败案例）
6	演示环境	可现场演示的预配置环 拟数据

文档中如有无法演示的环节（如需要特定硬件），必须提供录屏和代码走查作为补充证据。

十、关键概念与实践引导

本节面向"具备一定理论但实践经验欠缺"的同学，把课题里反复出现的几个关键概念落到"具体怎么做"上。理解这些能帮你避免踩坑，也能让上面的需求读起来更顺。

10.1 小参量模型与离线推理

- **参数量**：模型的"参数" (parameter) 数量决定其能力与资源占用，7B $\approx$ 70 亿参数。课题要"小参量"，是因为客户机（尤其利旧旧硬件、纯 CPU 环境）跑不动大模型，且离线场景没有云端大模型可用。
- **为何要离线部署**：正常用大模型是调云端 API（如 `api.openai.com`），但客户断网 + 不接受公网 AI。所以要把模型文件拷进内网、在本地起一个推理服务，让工具连本地服务而非公网。
- **三种本地推理服务（任选其一）**：
 - **Ollama**：最易上手，用 GGUF 格式模型文件，`ollama create` 可从本地文件离线导入，自带 OpenAI 兼容 API。适合快速跑通。
 - **vLLM**：吞吐高、适合有 GPU 的环境，用 HuggingFace 格式 (safetensors) 模型文件。
 - **llama.cpp**：轻量，纯 CPU 也能跑，适合资源极度受限。
- **模型文件怎么离线进去**：在有网环境下载好模型文件 (GGUF / safetensors)，通过 U 盘或内网文件服务器拷到客户机，导入本地推理服务。这就是红线 1 说的"部署阶段一次性离线导入"。
- **资源差异**：7B 模型量化后约 4-5GB 显存 / 内存，纯 CPU 推理速度较慢；1.5B-3B 纯 CPU 也能流畅跑。选型时先看目标机有没有 GPU、多少内存，再定模型规格（对应 REQ-A-01 的资源预检）。

10.2 OpenAI 兼容 API 是什么、为什么用它

- 它是一套接口规范：只要某个推理服务实现了 `/v1/chat/completions` 这类端点，你就能用同一套 SDK（如 `openai` Python 库）去调它，只是把 `base_url` 指向本地服务地址（如 `http://localhost:11434/v1`）。
- **好处**：你的代码不绑定某个云端模型，换本地模型只改 `base_url` 和 `model` 两个配置，不用改业务代码。这也正是技术约束 8.1 要求 `base_url` / `model` 可配的原因。

10.3 Prompt 注入与"关键路径不经 LLM"

- **Prompt 注入**：用户或日志里可能含恶意文本，诱导 LLM 做出错误判断（如让 LLM 认为"用户已确认执行"）。
- **防线原则**：人工审核确认、审计日志写入、危险命令拦截这三条**安全关键路径的判定不能由 LLM 完成**，必须用硬编码逻辑（如 CLI 的 `[y/N]` 输入、专用 HTTP 接口、危险命令正则匹配）。LLM 只能"建议"，"决定权"在硬编码逻辑里。

10.4 CLI / TUI 实践

- 无图形界面的服务器上，友好的命令行体验可以参考这几个库：**Rich**（表格、颜色、进度条）、**Textual**（完整 TUI 应用）、**Prompt Toolkit**（交互式输入、自动补全）。三者都是 MIT/BSD/Apache 协议，可放心使用。
- 记得提供 `--help` 和命令补全，降低运维人员记忆负担。

10.5 工程质量提醒

- 这是个工程落地课题，**做得完、不出事**比花哨重要：先把端到端流程跑通（哪怕每个环节都很基础），再逐项加深。
- 错误处理不能"吞"：采集失败、模型调用失败、检测报错都要明确提示，不能静默忽略（对应多条需求的验收标准）。
- 不要硬编码：命令建议用占位符、外网地址不写死、模型路径不写死——这也是"离线可用"和"可维护"的共同前提。